# Assignment 3

Name: Priyanka Sanjay Jagtap

Class: BEIT

Roll Number: 407I085

## Code and Output:

```
import os
import re
from collections import defaultdict
```

```
# --- Function to preprocess text (remove punctuation, lowercase) ---
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z0-9\s]', '', text)
    return text.split()
```

```
def build_inverted_index(folder_path):
    inverted_index = defaultdict(list)
    files = [f for f in os.listdir(folder_path) if f.endswith('.txt')]

    for idx, file in enumerate(files):
        with open(os.path.join(folder_path, file), 'r', encoding='utf-8') as f:
            words = preprocess(f.read())
            for position, word in enumerate(words):
                inverted_index[word].append((file, position))

    return inverted_index, files
```

```python
def search_word(inverted_index, query):
    query = query.lower()
    if query in inverted_index:
        return inverted_index[query]
    else:
        return []
```

```python
def display_index(inverted_index):
    print("\n--- Inverted Index ---")
    for word, occurrences in inverted_index.items():
        print(f"{word}: {occurrences}")
```

```python
if __name__ == "__main__":
    folder = "documents"  # folder containing your text files
    inverted_index, files = build_inverted_index(folder)

    # Display complete index
    display_index(inverted_index)

    # --- Query input ---
    while True:
        query = input("\nEnter a word to search (or 'exit' to quit): ").strip()
        if query.lower() == 'exit':
            break
        results = search_word(inverted_index, query)
        if results:
            print(f"\n'{query}' found in:")
            for file, pos in results:
                print(f"  - {file} (position {pos})")
        else:
            print(f"\n'{query}' not found in any document.")
```

```
--- Inverted Index ---
artificial: [('doc2.txt', 0), ('doc1.txt', 6)]
intelligence: [('doc2.txt', 1), ('doc2.txt', 14), ('doc1.txt', 7)]
focuses: [('doc2.txt', 2)]
on: [('doc2.txt', 3)]
creating: [('doc2.txt', 4)]
systems: [('doc2.txt', 5), ('doc1.txt', 15)]
capable: [('doc2.txt', 6)]
of: [('doc2.txt', 7), ('doc1.txt', 5)]
performing: [('doc2.txt', 8)]
tasks: [('doc2.txt', 9)]
that: [('doc2.txt', 10), ('doc1.txt', 8)]
normally: [('doc2.txt', 11)]
require: [('doc2.txt', 12)]
human: [('doc2.txt', 13)]
machine: [('doc1.txt', 0)]
learning: [('doc1.txt', 1)]
is: [('doc1.txt', 2)]
a: [('doc1.txt', 3)]
field: [('doc1.txt', 4)]
uses: [('doc1.txt', 9)]
statistical: [('doc1.txt', 10)]
techniques: [('doc1.txt', 11)]
to: [('doc1.txt', 12), ('doc1.txt', 18)]
...
ability: [('doc1.txt', 17)]
learn: [('doc1.txt', 19)]
from: [('doc1.txt', 20)]
data: [('doc1.txt', 21)]
```

*Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...*